

공세민

Se Min Kong

로봇과 소프트웨어를 연결하는 개발자

안녕하세요. 공세민입니다.

숭실대학교에서 소프트웨어를 전공했습니다.

현재 SSAFY Robotics Track에서 공부하고 있습니다.

센서 입력부터 제어 명령과 실제 동작까지 연결하며,
동작 조건과 실패 경계를 함께 살펴봅니다.

semin1224@gmail.com

github.com/SeMinKong



현재

SSAFY Robotics Track
2026.01 - 현재

학력

숭실대학교
소프트웨어학부
2020.03 - 2026.02

관심 분야

Computer Vision
Robotics / Physical AI

거주지

Suwon,
Republic of Korea

학력과 관심 분야

학력·교육

숭실대학교 소프트웨어학부

2020.03 - 2026.02

소프트웨어 공학사

인공지능·빅데이터 전공

빅데이터 융합전공

SSAFY Robotics Track

2026.01 - 현재

삼성청년SW·AI아카데미 교육생

로보틱스, Computer Vision, ROS 2와

하드웨어·소프트웨어 통합을 공부합니다.

병역 2021.05 - 2022.11

육군 전술 통신 장비 운용·정비 / 병장 전역

어학 OPIc IH / 2027.10까지 유효

관심 분야

Computer Vision & Robotics

지금은 Computer Vision과 ROS 2, 로봇 제어를 중심으로 배우고 있습니다. 소프트웨어가 실제 장치와 만나는 로보틱스와 Physical AI에 관심이 있습니다.

Simulation

Isaac Sim과 Isaac Lab은 현재 학습 중인 도구입니다. 센서 오차, 통신 지연, 접촉과 마찰처럼 시뮬레이션과 실제 환경 사이에서 달라지는 조건에 관심을 두고 있습니다.

Local AI & Software

Ollama와 llama.cpp 등 로컬 AI 도구를 살펴보고 있습니다. Python과 C++를 사용하며, Ubuntu, Git, Docker와 같은 개발 환경도 함께 다룹니다.

프로젝트

장비 연동과 실물 제어를 중심으로, 입력과 출력의 품질을 다룬 작업을 소개합니다.

쪽	프로젝트	역할	핵심 판단
4 - 7	THING	6인 팀 / 구동·기구 통합	장력·권취 방향 / 초기 위치와 정지
8 - 11	AQIS	2인 팀 / 팀장·서버·장비 통합	검출 시점과 집기 순서 / 예외 입력
12 - 13	Briefit	6인 팀 / AI 담당	입력 정제 / 반복 제거와 정보 보존
14 - 15	Brain MRI	개인 / 전처리·학습·통합 추론	학습 형식 변환 / 평가 데이터 분리
16 - 17	Prompt Generator	개인 / 대화 서버·상태 관리	영역별 대화 상태 / 수정 흐름
18 - 19	Alkkagi.io	개인 / 클라이언트·서버·물리	서버 기준 상태 / 입력 검증

THING · 텐던 로봇 핸드

기간

2026.07 - 08

팀·역할

6인 팀

모터 제어·기구 통합

사용 도구

DYNAMIXEL / U2D2

모터 점검·제어 스크립트

팀 기술

ROS 2 / MediaPipe

OpenCV / Jetson

Raspberry Pi 5



모터 고정부·스플·텐던을 연결한 통합 조립 상태

프로젝트 개요

카메라에서 읽은 손동작을 7개 논리축 명령으로 바꾸고 텐던 로봇손을 구동합니다. 엄지는 3축, 나머지 네 손가락은 각각 1축으로 다룹니다. 팀 시스템은 21개 손 landmark를 인식하고, 명령 중재와 guard를 거쳐 모터를 구동하도록 구성했습니다.

담당 작업

모터 통신 환경과 U2D2 연결, 7개 모터 점검·제어 스크립트, 전완부 아크릴 고정부 제작과 스플·텐던 통합을 맡았습니다. 통신 확인, 개별 제어, 실제 조립을 각각 점검할 수 있도록 작업을 나눴습니다.

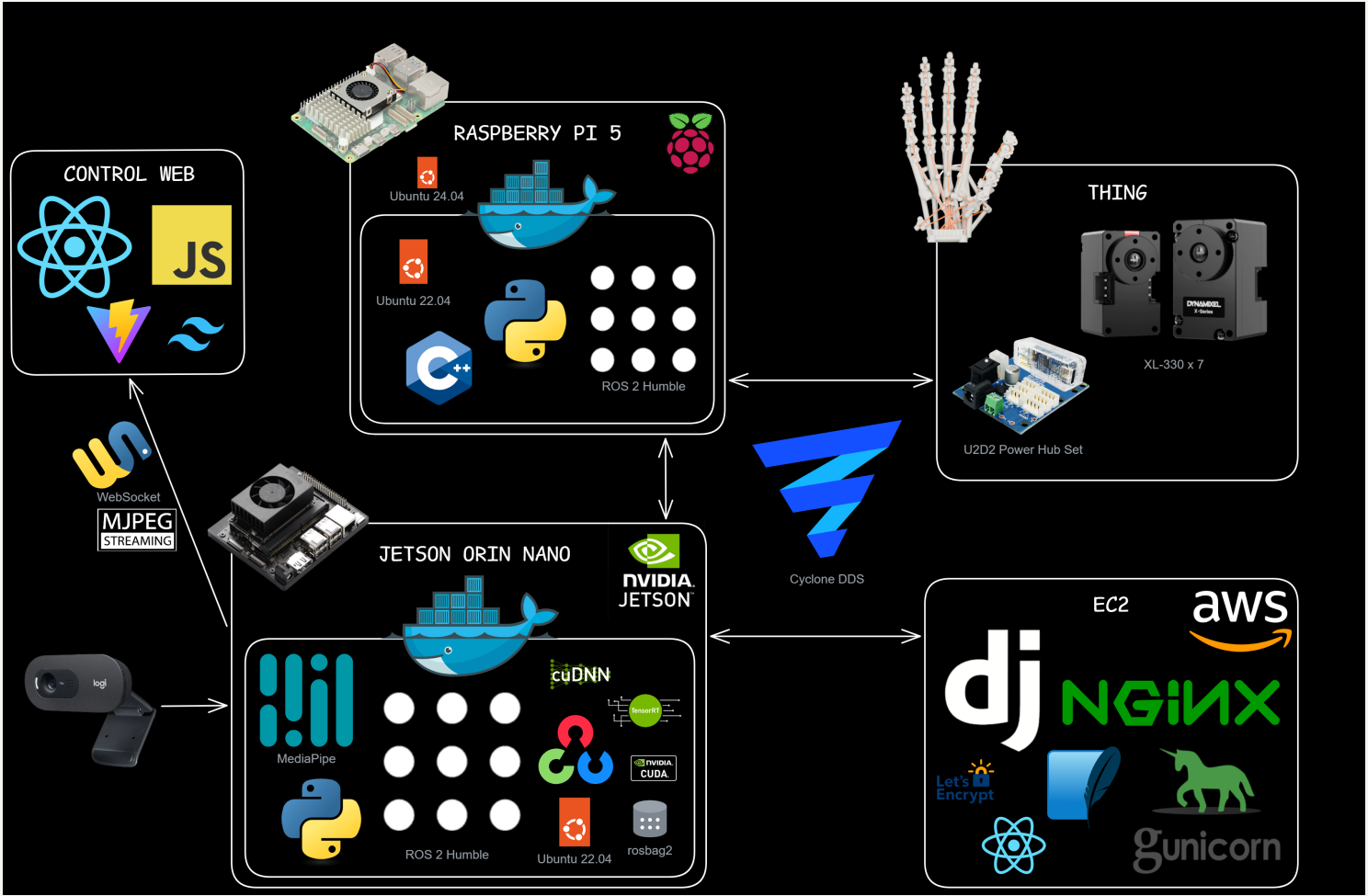
협업 범위

손동작 인식, ROS 2 명령 중재·guard, 관제와 데이터 기록은 팀 전체의 결과입니다. 이 사례에서는 직접 수행한 구동·기구 작업을 중심으로 설명합니다.

SSAFY 공통 프로젝트 우수상 / 2026.08.10

THING · 시스템 구조

최종 발표자료의 원본 아키텍처 / 장치별 실행 환경과 통신 연결



팀 시스템의 연결

Jetson에서 손동작을 인식하고 Raspberry Pi의 ROS 2 제어 경로를 거쳐 로봇손을 구동합니다. 관제 화면은 영상과 상태를 받습니다.

직접 맡은 연결

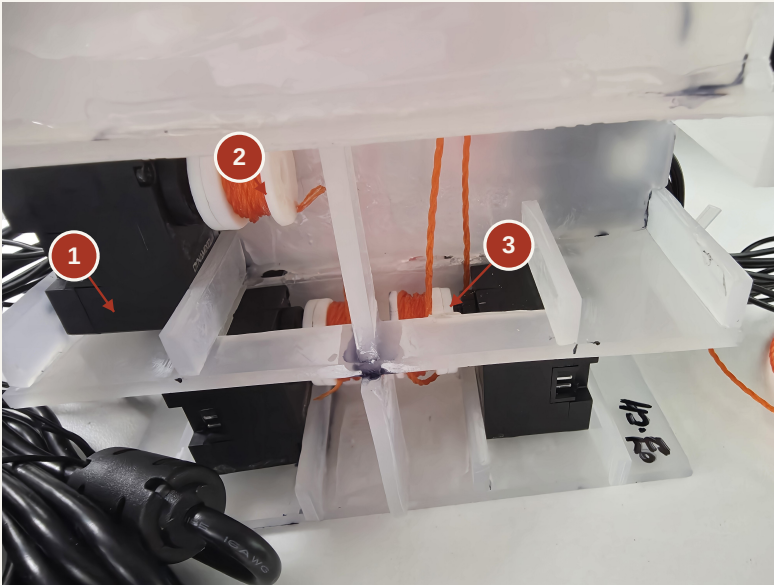
모터 통신 환경, 점검·제어 스크립트, 아크릴 고정부와 스푼·텐던 조립을 맡았습니다. 인식·관제·안전 로직은 팀의 협업 결과입니다.

기록 경로

Cyclone DDS는 내부 ROS 2 통신에 사용합니다. Jetson에서 EC2로 보내는 기록은 HTTPS 업로드입니다.

THING · 기구 편차와 제어 조건

같은 모터 위치 명령도 텐던 장력과 권취 방향에 따라 실제 손가락 자세가 달라집니다.



구동부 내부: 모터·스풀·텐던 연결



전완부 모터 고정부

- 1 모터 위치와 손 자세**
위치 명령과 실제 굽힘을 구분합니다.
- 2 스푼 권취 방향**
감기는 방향이 텐던의 당김을 바꿉니다.
- 3 축별 텐던 장력**
동일 목표값만으로 자세를 보장하지 않습니다.
- 4 고정·재장착 조건**
체결 방향과 텐던 경로를 사진으로 남겼습니다.

이동 명령의 토크 순서

이동 명령에서는 현재 위치 읽기 → 목표값 기록 → 토크 ON 순서를 적용합니다. 이전 목표값으로 갑자기 이동하는 것을 막기 위한 점검 코드입니다.

원위치와 정지

다회전 위치에서 가장 가까운 중앙각 좌표를 선택합니다. 키보드로 개별·전체 모터의 토크를 끄는 Torque OFF 경로도 마련했습니다.

기구 통합의 확인 범위

아크릴 고정부를 제작하고 모터·스풀·텐던을 통합했습니다. 축별 끝점 보정과 최대 가동 범위의 간섭 검증은 남은 과제로 정리했습니다.

THING · 실물 시연과 재현성



엄지와 손가락으로 원통형 물체를 감싸 쥐는 동작

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 / THING

[상장 보기](#)

시연한 동작

사람의 손동작 모방, 손가락 순차 동작, 캔과 부드러운 물체 파지를 시연했습니다. 모터 명령이 기구를 거쳐 실제 손가락 동작으로 이어지는 것을 확인했습니다.

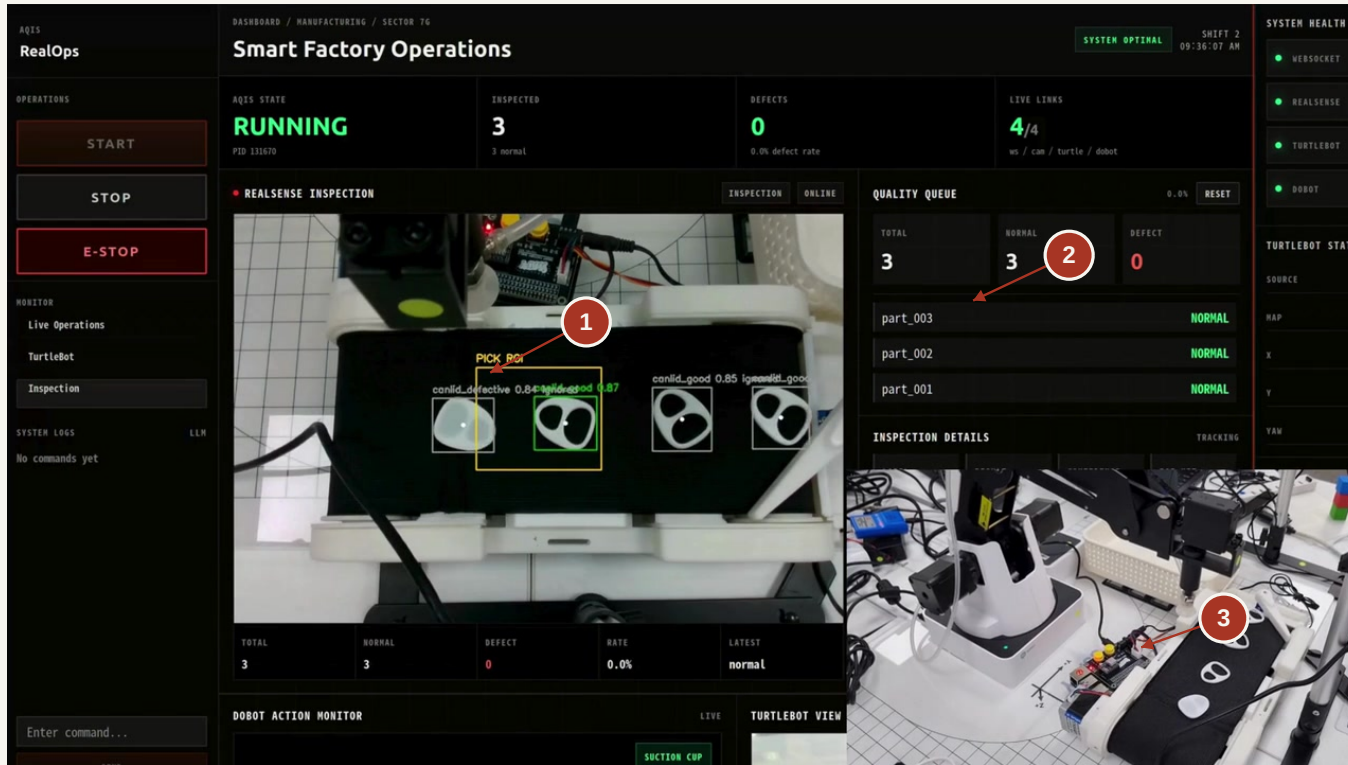
재장착을 위한 기록

스풀·텐던 경로를 사진으로 남겨 정비·재장착 시 참고하도록 했습니다. 작업일지는 체결 방향 확인과 축별 장력·가동 범위의 추가 검증을 구분합니다.

검증 범위 영상은 개별 동작의 시연 근거입니다. 보정 전후 편차와 재조립 후 반복성, 물체별 파지 성공률은 정량 결과가 없습니다.

AQIS · 스마트 팩토리

검사, 컨베이어, Dobot과 RealOps 관제를 연결한 스마트 팩토리 시스템



검출 영역, 작업 대기열과 실제 장비의 동작을 함께 보는 RealOps 관제 화면

직접 맡은 일 React 관제, FastAPI·WebSocket, ROS 2 연결, 장비 adapter와 Dobot 집기 시퀀스를 담당했습니다. LLM 명령과 키워드 fallback도 연결했습니다.

협업 팀원은 모델 학습·Roboflow·CAD·시뮬레이션을 맡았습니다.

2026.05 기획 / 06 본 개발

2인 팀 · 팀장

Full-stack & Robot Integration

1 검사 대상과 검출 영역

카메라에서 인식한 대상의 위치와 집기 영역을 확인합니다.

2 판정과 작업 대기열

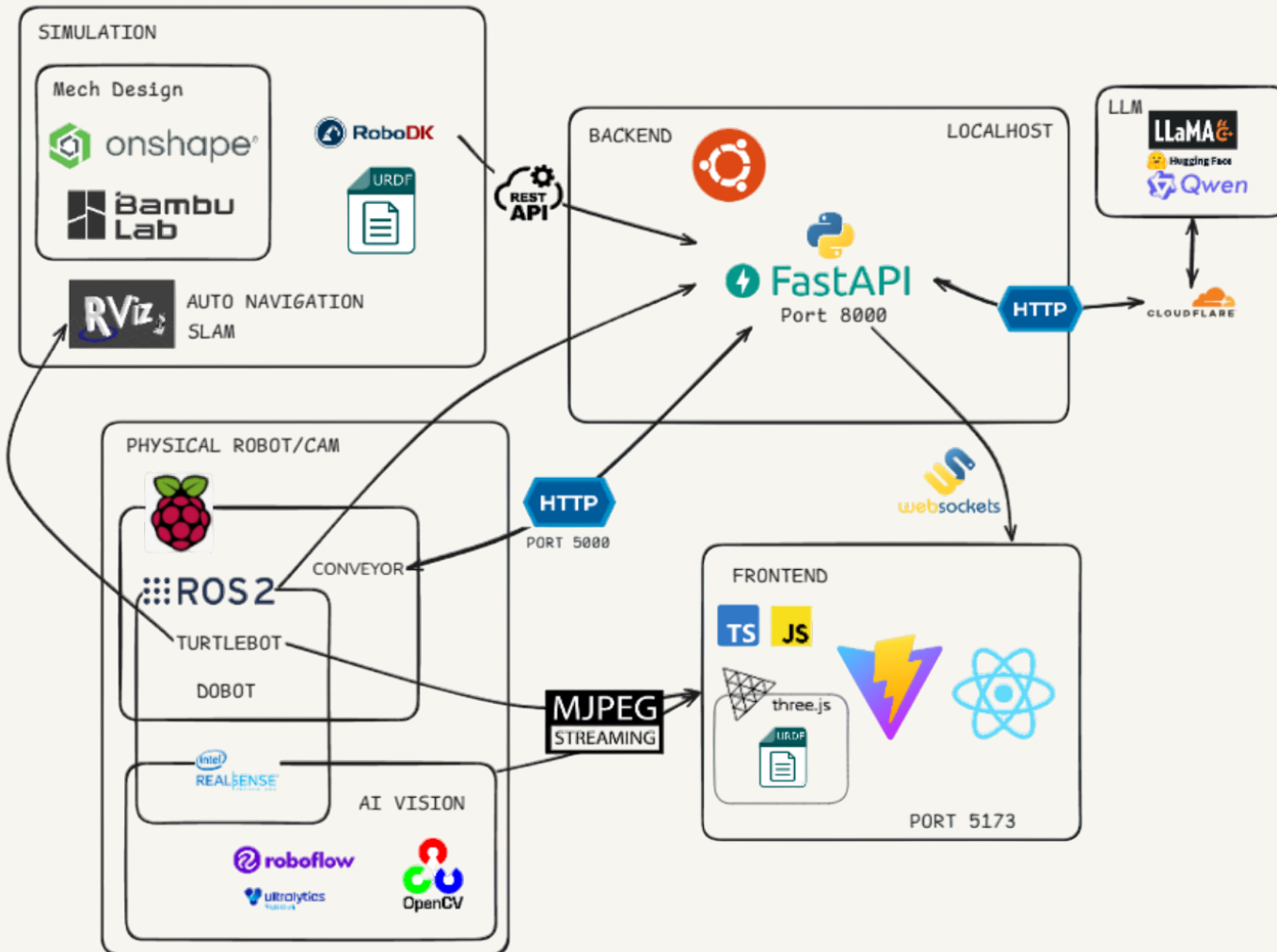
검사 결과와 처리할 작업을 관제 화면에서 확인합니다.

3 로봇·컨베이어 동작

물체를 옮기는 장비의 동작을 검사 화면과 함께 봅니다.

AQIS · 장비 사용 전 통합 경로 확보

초기 일정의 제약을 Mock/Real adapter와 공통 관제 인터페이스로 다뤘습니다.



제약: 마지막 3일

초기 계획에서 실물 장비 사용은 마지막 3일로 제한되었습니다. 앞선 기간에 관제·API·WebSocket 흐름을 먼저 연결하는 전략을 정했습니다.

선택: 연결부 분리

Mock 응답으로 개발한 관제 아래에 실제 장비 adapter를 연결했습니다. 서버의 공통 API와 상태 이벤트를 유지하면서 장비별 연결을 분리했습니다.

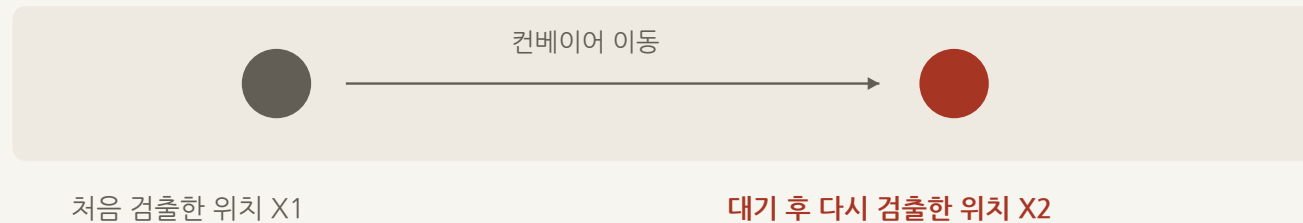
통합 범위

컨베이어·Dobot·검사 관제를 연결했습니다. 그림의 TurtleBot 자동 임무는 확장 설계이며, 구현은 SLAM·카메라·상태 관제까지입니다.

AQIS · 집기 좌표 갱신

문제: 컨베이어 위 대상이 이동하면, 로봇이 도착할 때 최초 검출 좌표는 이미 오래된 정보입니다.

검출 시점에 따른 위치 변화



좌표 재취득

정지 명령 뒤 대기 상태로 전환합니다. 이후 검출에서 깊이 좌표를 우선 사용해 집기 명령을 구성합니다.

1

검출 수신

작업 조건 확인

2

> 정지 요청·대기

설정 시간 대기

3

> 좌표 재취득

후속 검출·깊이 좌표 우선

4

> 집기·분류·재개

스크립트 정상 종료 시 재개

중복 집기 방지

객체 ID, 검출 영역 겹침과 중심 거리로 중복 여부를 판단합니다. 작업 중에는 집기 명령의 중복 실행도 막습니다.

물리 완료와 구분

대기 시간은 실제 정지 확인을 대신하지 못합니다. 재개 조건도 스크립트 종료 코드이며 물체의 파지 성공을 센서로 확인한 결과는 아닙니다.

AQIS · 예외 입력과 검증 범위

소프트웨어의 제어 조건과 실물 공정의 완료 여부를 나눠 확인합니다.

입력 조건	구현한 제어	확인 범위
같은 대상의 반복 검출	객체 ID 또는 영역·중심 거리로 중복 집계를 억제합니다.	원본 Mock 테스트와 후속 재현에서 동작 확인
관제가 정지된 상태	검출을 받아도 통계를 갱신하거나 집기 명령을 보내지 않습니다.	원본 Mock 테스트와 후속 재현에서 동작 확인
timestamp가 있는 오래된 검출	준비 시점 이전 또는 허용 연령을 넘은 검출을 걸러냅니다.	정지 전 시각을 넣은 Mock 사례에서 차단 확인

아직 보강할 실패 경로

timestamp가 없으면 신선도 검사를 통과하고, 깊이가 없으면 고정 좌표를 사용합니다.
자동 흐름은 정지 명령의 실패 응답으로 다음 단계를 차단하지 않습니다.

공정 성능의 확인 범위

장비 시연은 검사·집기·분류의 연결을 보여줍니다. 반복 성공률과 사이클 시간, 통신 단절 후 복구는 정량 검증 자료가 없습니다. 좌표 변환의 기대값 불일치도 남아 있습니다.

검증 출처: 원본 테스트 + 후속 Mock 재현. 실물 분류 성능이나 당시 팀의 반복 시험 결과로 해석하지 않습니다.

Briefit · 뉴스 요약과 출력 품질

2025년 본인 KoBART 작업 / 입력 정제와 생성문 후처리를 분리해 구현했습니다.

기간

2025.05 - 09

팀·역할

6인 팀 / AI 2인

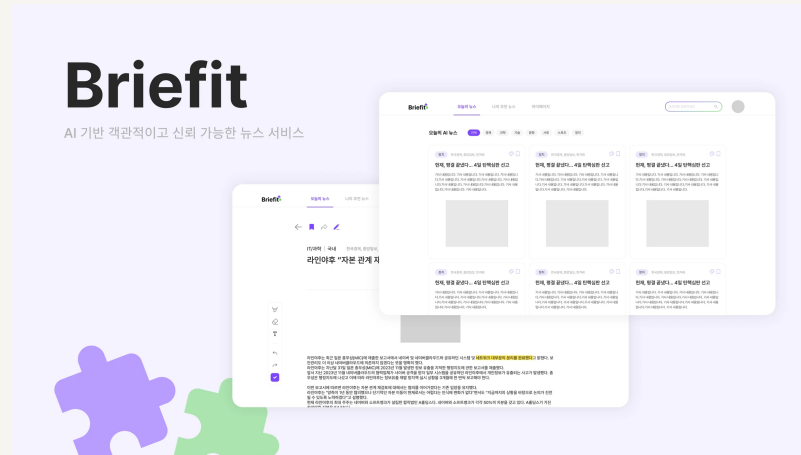
본인: AI 담당

사용 기술

Python / aiohttp

BeautifulSoup

Transformers / KoBART



뉴스를 모아 읽고 요약을 확인하는 서비스
제품 UI는 팀의 협업 결과입니다.

직접 구현한 범위

기사 수집 필터·중복 제거와 KoBART 데이터 분할·학습·생성·ROUGE
평가 스크립트를 작성했습니다. 생성 뒤에는 반복 종결문을 정리하는
규칙을 추가했습니다.

품질 판단 반복 제거만으로 요약 품질을 보장할 수는 없습니다. 정상 문장의
삭제와 핵심 정보 누락을 함께 평가해야 합니다.

프로젝트 수상

2025 IT대학 소프트웨어 공모전 금상

2025.08.18 / Briefit

[상장 보기](#)

제15회 숭실 캡스톤디자인 경진대회 장려상

2025.10.01 / Briefit

[상장 보기](#)

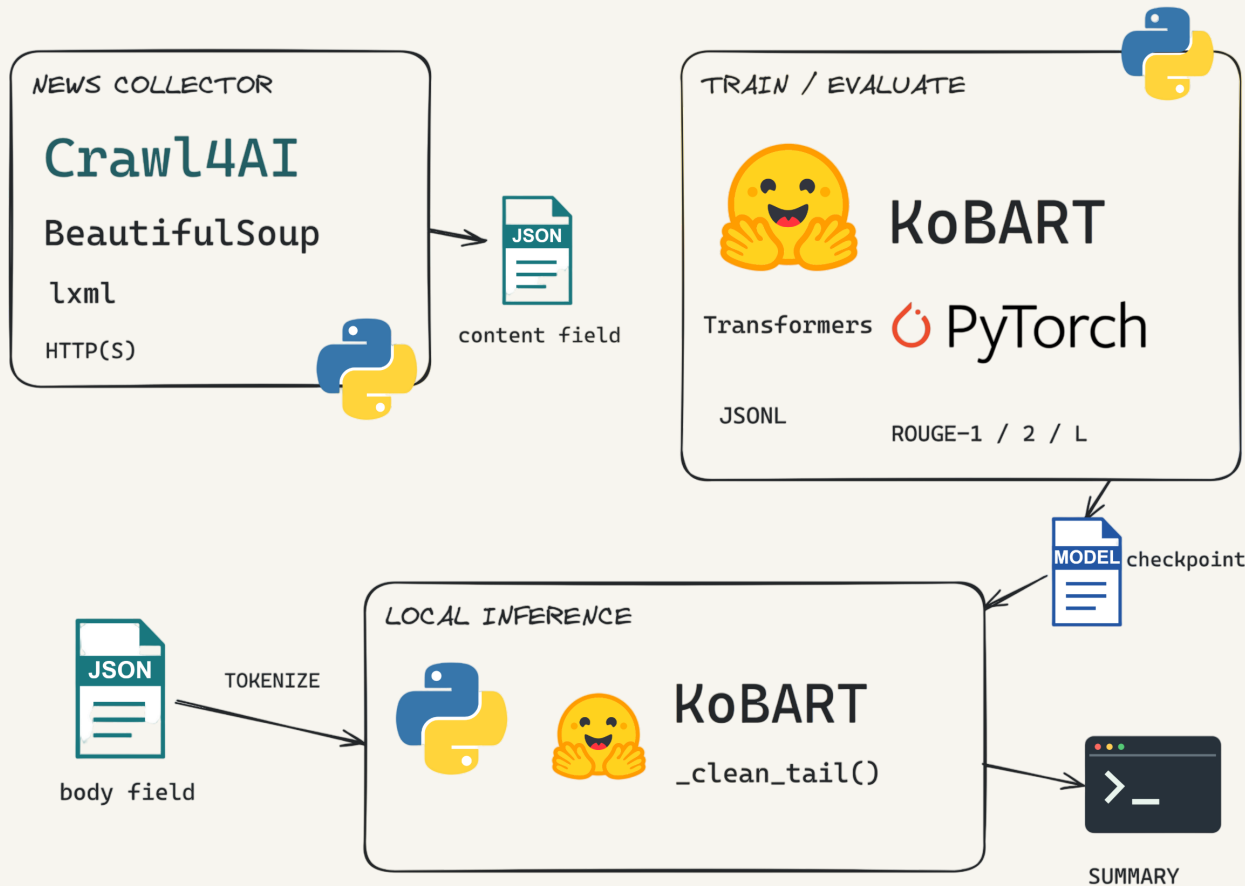
2025 IT 프로젝트 프로리그 장려상

2025.11.22 / Briefit

[상장 보기](#)

Briefit · 반복 제거와 정보 보존

수집 필터 → 모델 생성 → 출력 후처리 / 각 단계의 오류와 평가 범위를 구분합니다.



입력 정제

중복 URL·덧글 URL·짧은 본문을 수집 단계에서 제외합니다. 수집 파일의 content와 추론 입력의 body는 필드 매핑이 필요합니다.

출력 후처리의 선택

반복되거나 짧은 끝문장을 규칙으로 제거합니다. 짧지만 유효한 문장도 삭제될 수 있어 반복 감소와 정보 손실을 함께 살펴야 합니다.

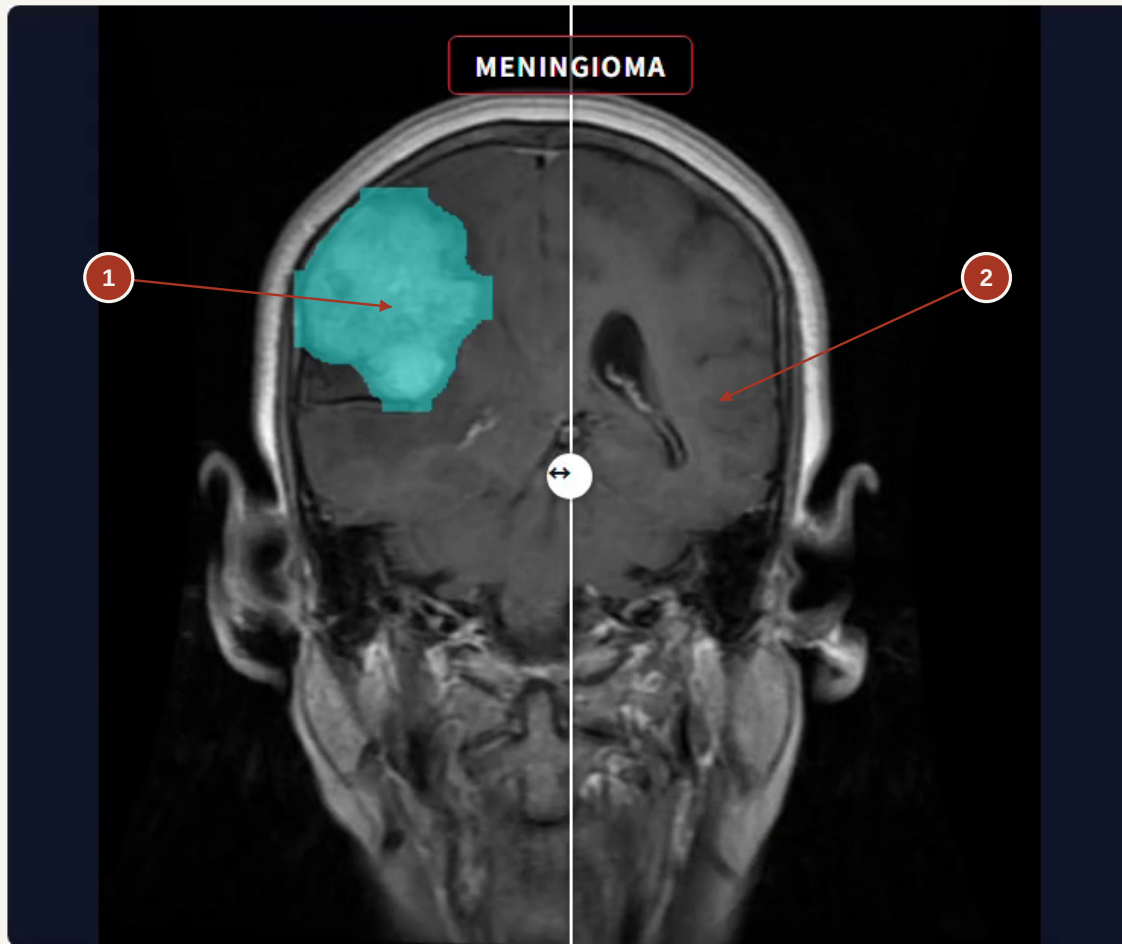
평가 범위

ROUGE 코드는 후처리 전 생성문을 평가합니다. 실제 점수 로그와 후처리 전후 품질 결과는 확인되지 않아, 정보 보존 효과를 성과로 제시하지 않습니다.

데이터 구성: 학습 2,819건 / 검증 352건 / 테스트 353건. 저장된 분할이며 모델 성능 수치가 아닙니다.

Brain MRI · 종양 분류와 영역 분할

개인 프로젝트 / 2026.02 - 04 / Python · PyTorch · YOLO11 · OpenCV



① 왼쪽: 예측 영역 표시 ② 오른쪽: 원본
분류 결과는 수막종(Meningioma)으로 표시됩니다.

구현 범위

분류와 분할을 각각 수행하는 두 YOLO11 모델의 학습·추론 경로를 구성했습니다. 분류 범주와 분할 위치를 하나의 이미지에서 함께 볼 수 있게 했습니다.

입력 형식의 문제

원본 마스크를 그대로 쓰는 대신 분할 학습에 필요한 polygon label로 바꿔야 했습니다. 이진화, 형태학 처리, 외곽선 추출과 좌표 정규화를 연결했습니다.

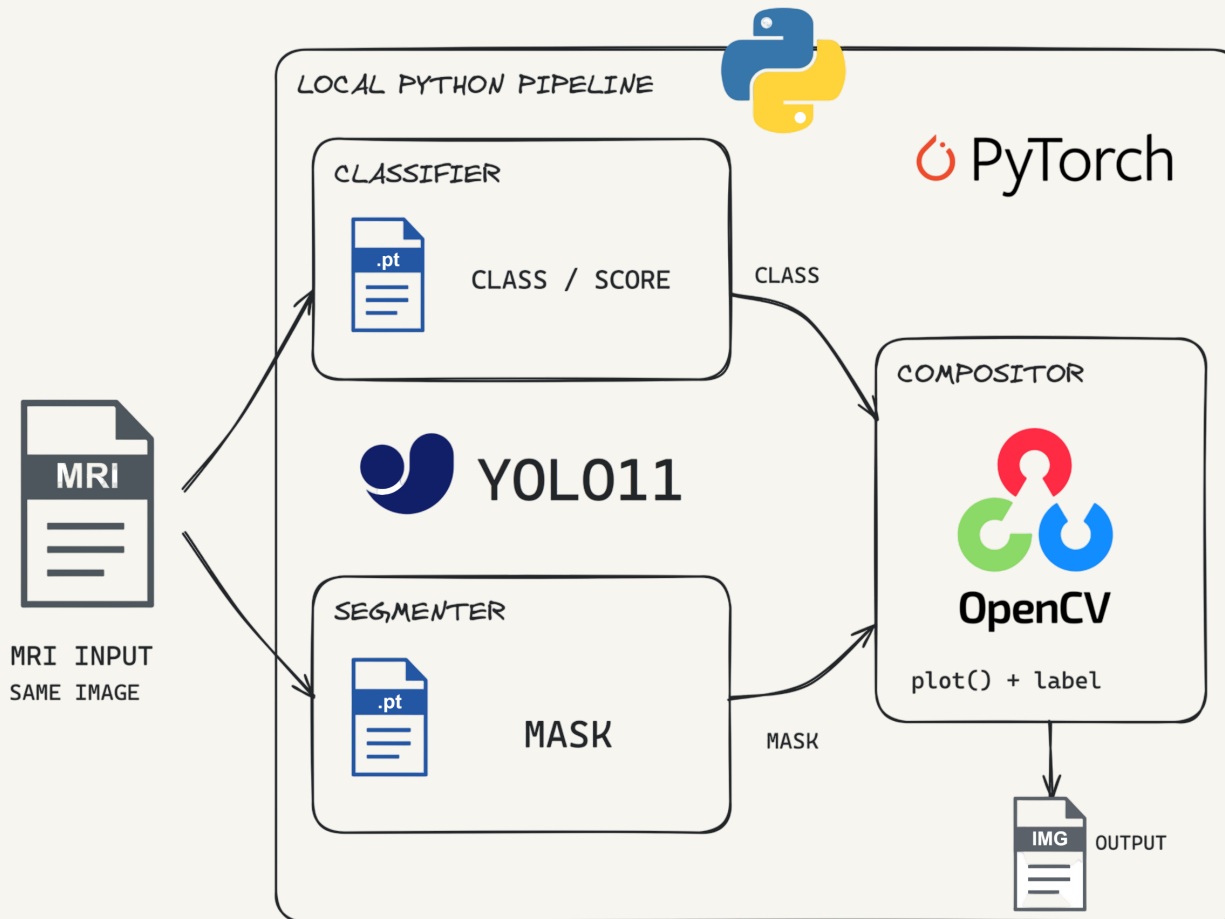
본인 범위

데이터 변환, 학습 설정과 통합 추론 코드를 작성했습니다. 두 모델을 독립적으로 학습하고, 결과를 한 화면에서 비교하도록 구성했습니다.

사용 목적 연구·학습용 프로젝트이며
임상 진단을 위한 검증은 수행하지 않았습니다.

Brain MRI · 분류·분할 모델 구조

같은 MRI를 두 모델에 각각 입력합니다. 한 모델의 결과를 다른 모델의 입력으로 사용하지 않습니다.



학습 형식 변환

이진화·형태학 처리·외곽선 추출로 마스크를 polygon label로 바꿉니다. 작은 영역 보존과 누락 여부는 실제 라벨로 확인할 대상입니다.

현재 평가 조건

학습 코드는 test 경로를 검증용 val에 연결합니다. 최종 성능을 판단하려면 학습·검증에서 사용하지 않은 별도 데이터로 평가해야 합니다.

데이터 해석

BRISC는 환자 식별 정보가 없어 환자 단위 독립성을 보장하기 어렵습니다. 비종양에도 병변이 포함될 수 있습니다. 임상 진단 검증은 수행하지 않았습니다.

학습 입력은 작업에 맞게 나눕니다. 분류는 폴더별 이미지, 분할은 마스크를 변환한 polygon label을 사용합니다.

Prompt Generator · 프로젝트 설계 도우미

아이디어를 영역별 질문으로 구체화하고 결과를 하나의 문서로 모으는 도구

기간

2026

팀·역할

개인 프로젝트

대화 서버·상태 관리

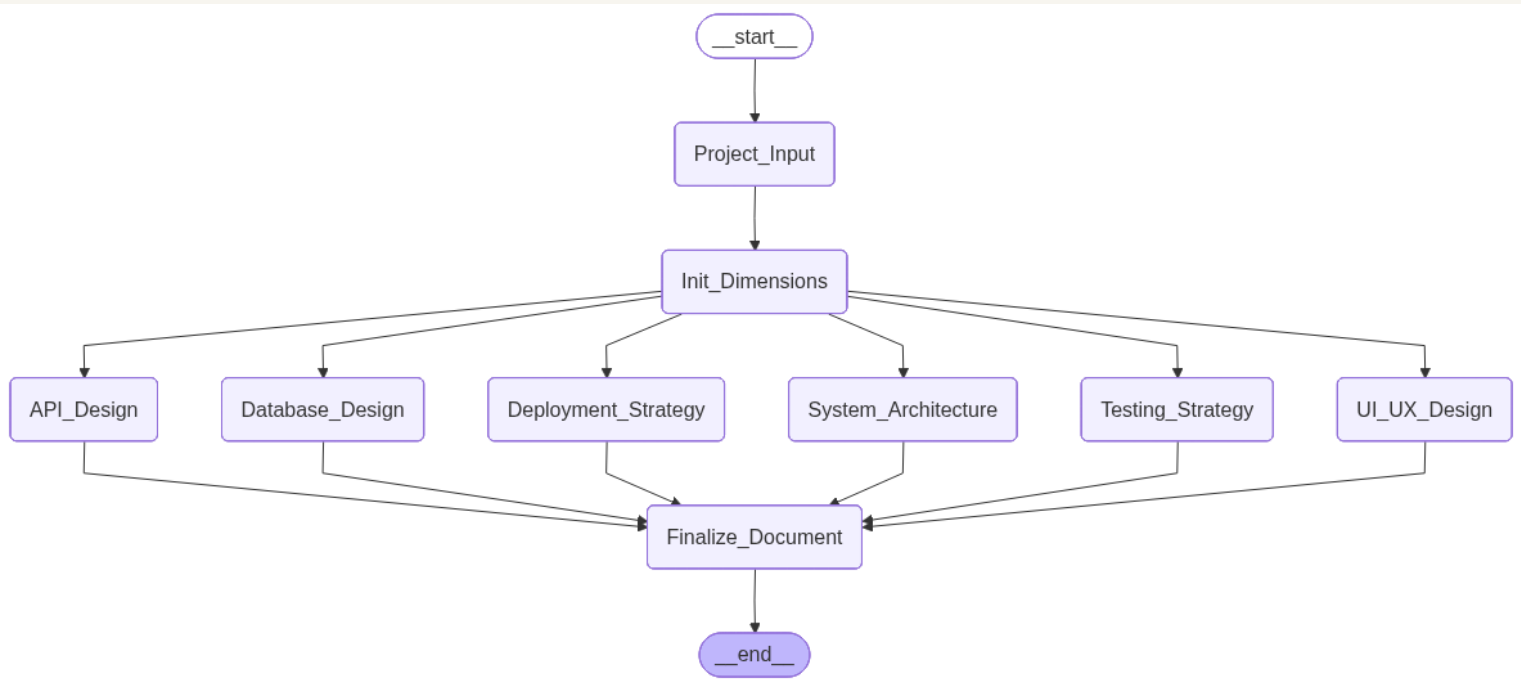
웹 UI

사용 기술

Python / FastAPI

WebSocket / LangChain

Solar Pro



프로젝트 아이디어를 여섯 설계 영역으로 나누는 원본 흐름도

프로젝트 개요

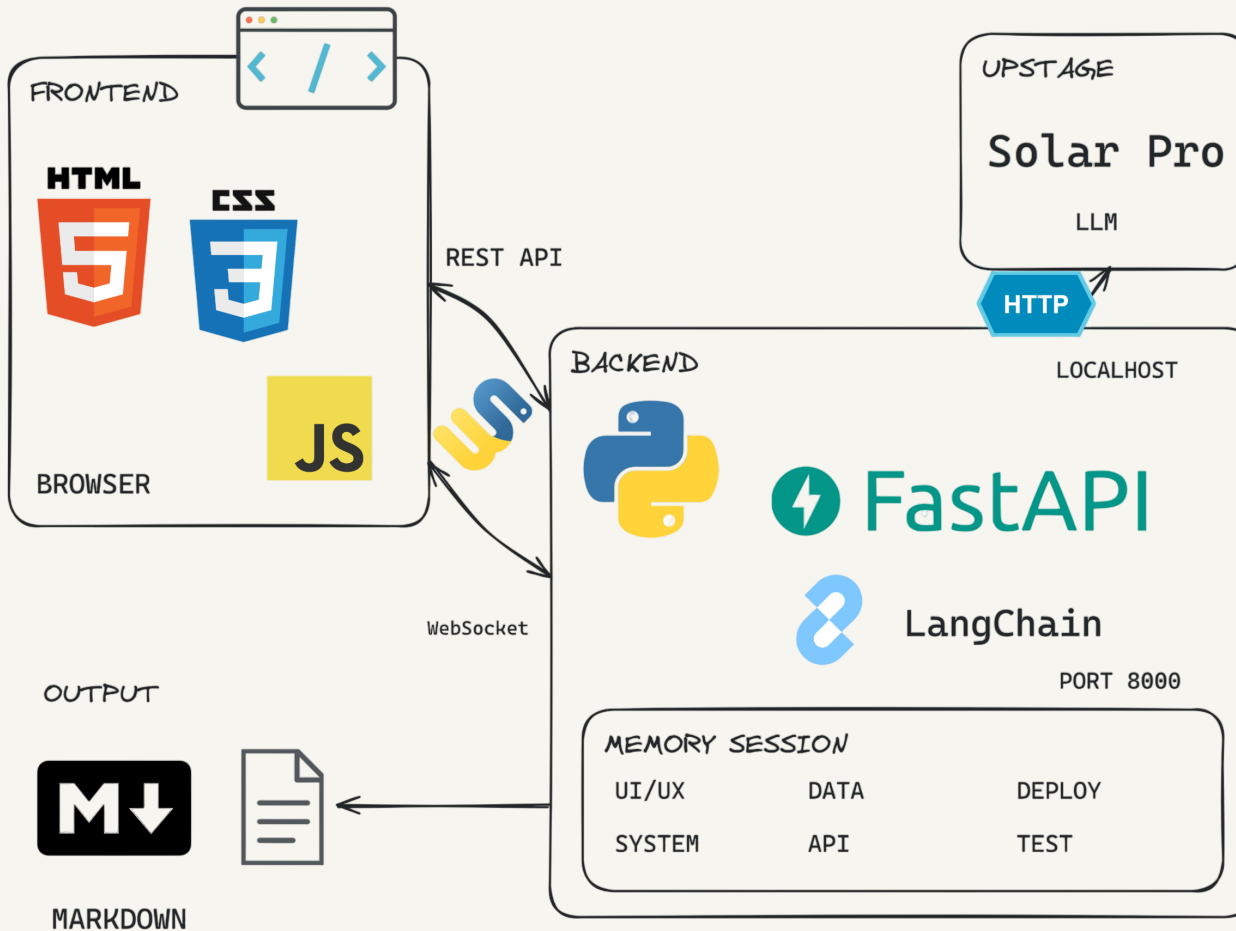
프로젝트 아이디어를 입력하고 화면·API·데이터 등 영역별 질문에 답하며 설계를 구체화합니다. 완료된 영역도 추가 대화로 수정할 수 있습니다.

직접 맡은 일

영역별 대화 이력과 진행 상태를 분리하고, 질문·상태를 웹 화면에 전달하는 서버와 결과를 문서로 모으는 흐름을 구현했습니다.

Prompt Generator · 영역별 설계 대화

개인 프로젝트 / 2026 / FastAPI · WebSocket · LangChain · Solar Pro



영역마다 대화 이력·진행 상태·결과를 따로 유지합니다.

문제와 선택

화면, 데이터와 배포 질문이 뒤섞이면 필요한 조건을 빠뜨리기 쉽습니다. 기본 여섯 영역의 첫 질문을 병렬로 시작하고, 각 영역의 답변을 따로 누적하게 했습니다.

구현 구조

브라우저는 WebSocket으로 질문과 상태를 받습니다. FastAPI는 세션별 대화를 관리하고 LangChain·Solar Pro 호출로 질문과 프롬프트를 생성하도록 구성했습니다.

수정과 복구의 경계

완료된 영역도 대화로 수정합니다. 세션은 연결 종료 시 삭제되며 재접속 복구는 없습니다. 요구사항 반영률이나 생성 품질의 평가 결과는 확인되지 않았습니다.

Alkkagi.io · 실시간 알까기

여러 플레이어가 같은 보드에서 돌을 밀어내며 대전하는 웹 게임

기간

2026.03 - 04

팀·역할

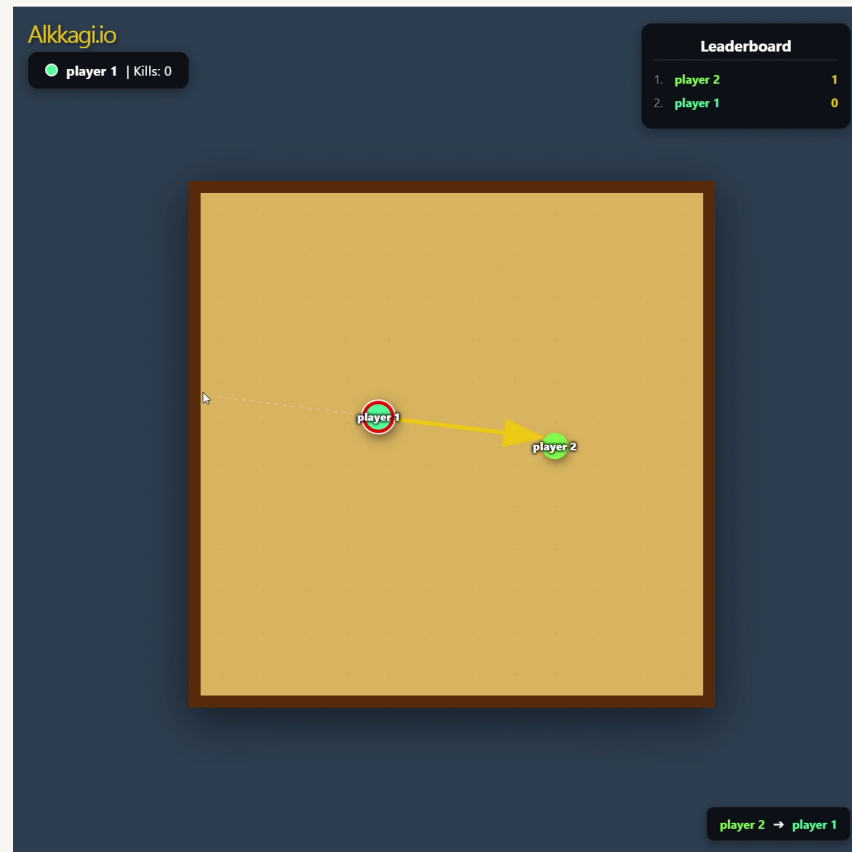
개인 프로젝트

웹 UI·서버·물리 계산

사용 기술

React / TypeScript

Node.js / Socket.IO



실제 플레이 영상의 한 장면 / 돌을 조준하는 화면

프로젝트 개요

돌을 드래그해 방향과 세기를 정하고 상대의 돌을 보드 밖으로 밀어냅니다. 점수가 오르면 돌의 크기와 질량도 변합니다.

직접 맡은 일

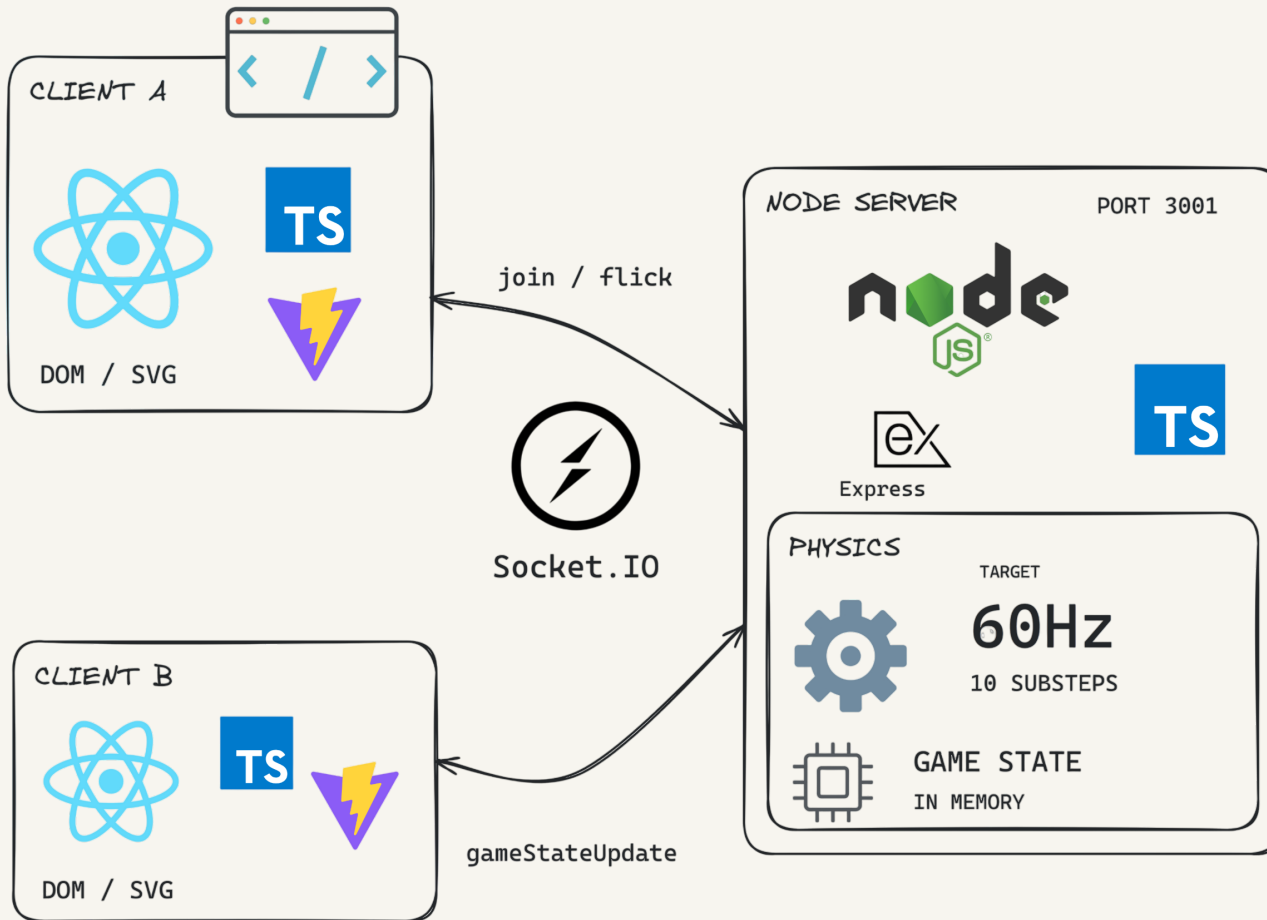
조준 UI, 실시간 통신과 서버 물리를 구현했습니다.
충돌·마찰·위치 보정을 TypeScript로 작성했습니다.

핵심 구현

서버가 게임의 기준 상태와 입력 제한을 관리하고, 모든 플레이어에게 같은 계산 결과를 전달하도록 구성했습니다.

Alkkagi.io · 서버 물리와 상태 동기화

개인 프로젝트 / React · TypeScript · Socket.IO / 기준 상태는 서버 메모리에 유지합니다.



기준 상태의 소유

클라이언트는 발사 입력을 보내고 서버가 위치·충돌·마찰을 계산합니다. 같은 계산 결과를 모든 플레이어에게 전달하도록 책임을 모았습니다.

서버에서 입력 검증

입력 간격과 발사 속도를 서버에서 제한합니다. 브라우저 UI의 제한만으로 게임 규칙을 유지하지 않도록 했습니다.

동기화와 지속성

메모리의 상태를 공유하고 연결 종료 시 플레이어들을 정리합니다. 네트워크 지연·재접속과 서버 재시작 후 복구는 별도 설계가 필요합니다.

서버가 유일한 기준 상태를 가지므로, 브라우저마다 다른 충돌 결과를 확정하지 않습니다.

후속 검증 동시접속자가 늘어날 때의 지연과 지속 프레임률은 별도로 측정해야 합니다. 서버 재시작 시 상태 복구도 추가 과제입니다.

연락처

Se Min Kong

공세민 · 소프트웨어 개발자

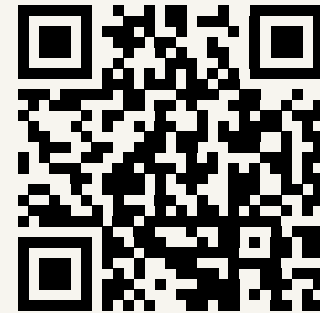
이메일

semin1224@gmail.com

GitHub

github.com/SeMinKong

웹 포트폴리오



seminkong.github.io/SeMinKong_Web/

프로젝트 영상과 상세 설명

이력서와 상장 전시